Suvion Systems - Environmental Monitoring and Sustainability Dashboard

Maritime vessel sustainability tracking made accessible

Alina Pak
April 5th, 2026

Introduction **3**
Executive Summary **3**
Biography **5**
Introduction & Background **6**
Goals & Proposed System **6**
Methodology & Design **8**
Scope **9**
System Architecture & Design **9**
Components 10
Data Flow 11
State Transitions 11
Storage 11
Privacy and Timing Enforcement Points 11
Success Metrics **12**
Sensor data 12
Test Procedure 12
Sample Size 12
Baseline 12
Log Types 12
Emissions Calculations 13
Test Procedure 13
Sample Size 13
Baseline 13
Log Types 13
Sustainability Reports 13
Test Procedure 13
Sample Size 14
Baseline 14
Log Types 14
Innovation & Differentiation **14**
1. Simulation-First Accessibility [Experimental] 14
2. Open-Source Stack for Zero Licensing Cost [Novel Application] 14
3. Automated IMO Emissions Calculation with Auditable Accuracy [Exploratory] 15
Risks & Mitigations **15**
Testing & Validation Plan **16**
Unit Tests — Emissions Engine 16
Unit Tests — Dashboard Widgets 17
Integration Tests — Data Ingestion 17
Integration Tests — Reporting API 18
End-to-End / System Tests 19
Deliverables 19

Schedule & Milestones **20**
Weeks 1-2: Setup Signal K, mock NMEA generator, basic dashboard skeleton. 21
Weeks 3-5: Emissions engine & real-time data pipeline. 21
Weeks 6-7: Reporting API, export formats 21
Weeks 8-9: Bug fixes, unit testing. Potentially: anomaly detection model and alert UI 21
Weeks 10-11: Integration testing, performance tuning, documentation. 22
Week 12: Demo prep, risk review, final polish. 22
Assumptions & Constraints **22**
Assumptions 22
Constraints 23
Ethics & Data Governance **23**
Data Retention 23
Data Deletion 23
Privacy Constraints 24
Compliance 24
Feasibility & Build Plan **24**
Why This Is Achievable in One Term 24
Canonical Terms **24**
Related Work **26**

Introduction

Suvion Systems helps small vessel operators easily track fuel consumption levels, monitor systems and calculate CO_2 emissions instead of having to manually log, calculate and analyze the data.

Suvion Systems addresses this by providing a single dashboard web application that reads real-time data from sensors using open-source software, calculates the environmental impact and produces standards-compliant sustainability reports.

Executive Summary

On a small yacht, the crew spends many hours (in addition to regular watch hours) manually tracking fuel and emissions using spreadsheets. This labour-intensive and error-prone process is done for fuel consumption reporting. The captain reviews the report and tries to analyze carbon intensity to see if the vessel complies with International Maritime Organization (IMO) standards on environmental contribution. To analyze the data, the captain spends hours calculating CO_2 emissions for the given report period. Manual monitoring led to delays in detecting high generator consumption and small water leaks, and by the time the issues were identified, the vessel had already breached environmental compliance standards for that charter period.

System pipeline: Sensor source → Signal K (NMEA normalisation) → Node.js Adapter (timestamping + persistence) → Emissions Engine (IMO calculation) → MongoDB (time-series store) → Reporting API (aggregation) → Next.js Dashboard (widgets + report builder). In simulation mode, a Python mock NMEA generator replaces physical hardware and drives the same pipeline end-to-end.

Currently, there are no alternatives to manual tracking for small vessel operators that can address the issues outlined above. The fix is a monitoring system that features a web dashboard with:

- real-time sensors in a widget view with multiple streams of data—fuel, water, energy, and waste
- automated CO_2 emissions calculations using IMO formulas and sensor-ingested data
- sustainability reporting with no compliance guarantee

In future (potential supporting work), the dashboard will offer:

- alert mechanisms for identifying non-compliant periods and detecting anomalies using lightweight machine-learning models
- sensor data stored in time-series storage
- regulatory compliance reporting

Suvion Systems provides a straightforward way for vessel environmental monitoring and sustainability tracking. The front-end will feature a dashboard page where users can view real-time sensor data, calculate CO_2 emissions using that data and generate sustainability and fuel consumption reports.

Behind the scenes, the dashboard will use an open-source software for collecting data from devices and sensors. The app will feature a simulation mode with sample NMEA0183 and NMEA2000 data, allowing users to explore the system without the need for hardware deployment.

During testing, Suvion Systems will be tested against:

- *Real-time Data Ingestion with Data Accuracy*. All simulated sensor streams appear in the dashboard within < 1 s of emission, with no data loss or corruption. 100 % match between simulated values and dashboard display; no unit mismatches.
- *CO_2 Emissions Calculation.* Calculated emissions differ from reference values (for a variety of fuel types and consumption rates) by < 0.5 %
- *Report Generation.* Reports are produced within 5 s, contain all required fields, and correctly flag non-compliant periods.
- *Simulation Mode.* Simulation can be started/stopped in < 2 s and the dashboard updates to match simulated conditions (cruising, anchoring, generator cycling).

Scope stays narrow on purpose.

- Suvion Systems only uses simulated vessel data with NMEA connections through Signal K (open-source central hub software). It provides configuration options for different data connections but the project doesn't test hardware installation with equipment on boats, only simulated data.
- Fuel consumption and sustainability reporting are restricted only to IMO standards on environmental contribution with publicly available formulas.
- There is no user authentication in place. Since the app's server is designed to potentially be installed on a physical vessel, it is assumed that the dashboard users are the crew members of the vessel.
- The application's FE is a single page dashboard
- Machine learning anomaly detection with time-series data is treated as future work, not part of the MVP release
- Compliance check is also treated as a potential future improvement and not part of the MVP. The system will generate the reports using saved sensor data and CO_2 emission calculations but not guarantee regulatory compliance

All results in this summary will be verified against reference values for CO_2 emissions for a variety of fuel types and consumption rates with multiple simulated conditions. Sensor accuracy will be evaluated over one week of recorded data. Real-time emissions will be demonstrated by logging socket and network connections.

Biography

I have developed scalable, full-stack applications and AI-powered tools based on real-time insights and automation. Most recently, at Navica AI, I contributed to a maritime intelligence platform that uses multi-agent AI workflows to enable predictive vessel management. In previous projects, I was responsible for delivering fintech software, designing custom AI agents with complex workflows, enhancing online pharmacies and telehealth platforms.

These experiences support building data ingestion, real-time dashboards, and analytics directly—all of which include designing data schemas and generating actionable insights. I am confident I can deliver simulation mode, sustainability reporting, and an open-source dashboard, complete with testable, reproducible results.

In this project, I will reuse my professional skills to implement the finalized design, build and debug the end-to-end full-stack systems prototype with a web dashboard, realtime capture of sensor data, emissions calculations and standards-compliant sustainability reports for small vessels. I will produce a test suite with unit tests for the web dashboard, integration tests for back-end services; design and run the evaluation scripts calculation errors, accuracy, and response time.

Introduction & Background

On small yachts the crew spends many hours logging fuel, generator use, water consumption and other operational data into spreadsheets. The resulting reports are labor-intensive, error-prone, and often incomplete.

Because the data is captured manually, captains must spend hours after each charter season to calculate CO_2 emissions using International Maritime Organization (IMO) formulas and compare the vessel's carbon intensity against regulatory thresholds. In practice this leads to late discovery of inefficiencies: excessive generator use, water leaks or fuel-spillage are only noticed after the fact, causing non-compliance with IMO environmental standards and loss of charter revenue.

There is a clear market gap: no affordable, open-source system exists that automatically captures sensor data, performs real-time emissions calculations, and produces standards-compliant sustainability reports for small vessels. The cost of commercial monitoring solutions is prohibitive (US $20-50 k plus installation), and spreadsheets lack the analytics needed to detect anomalies early.

Suvion Systems provides a single dashboard that displays widgets for each data stream, an emissions calculator and a report builder with date-range picker. All components are open-source and can be deployed on a single onboard computer or in the cloud.

Goals & Proposed System

Suvion Systems is a web-based dashboard that:

- Ingests real-time sensor data (fuel flow, generator current, water meters, temperature) via an open-source hub (Signal K).
- Calculates CO_2 emissions automatically using publicly available IMO formulas and the ingested sensor data.
- Generates sustainability reports (fuel consumption, CO_2 emissions, water usage) that can be exported for regulatory filing (but can't be used as standalone standards-compliant documents).
- Provides a simulation mode with sample NMEA0183/2000 streams so operators can test the system without hardware.

As part of future work and potential improvements, the system will:

- Detect anomalies (e.g., generator over-run, water leak) using lightweight machine-learning models and trigger alerts.
- Generate standards-compliant report with regulatory checks

The application is a single dashboard that displays widgets for each data stream, an emissions calculator and a report builder with date-range picker. All components are open-source and can be deployed on a single onboard computer or in the cloud.

goal	feature	test	evidence
use open-source software to automatically capture sensor data	real-time sensors in a widget view with multiple streams of data—fuel, water, energy, and waste	all simulated sensor streams appear in the dashboard within < 1 s of emission, with no data loss or corruption. 100 % match between simulated values and dashboard display; no unit mismatches.	logs for socket and network connections that captures: connection established, each message received with timestamp and payload, and disconnection events
perform emissions calculations	the system calculates CO_2 emissions automatically using publicly available IMO formulas and the ingested sensor data	calculated emissions differ from reference values (for a variety of fuel types and consumption rates) by < 0.5 %	a test suite (e.g. Jest, pytest) that feeds known inputs — a matrix of fuel types at varying consumption rates - into the calculation module and asserts that outputs match pre-computed reference values within 0.5%. The reference values should be derived directly from the IMO formulas, with the formula source cited in the test file so the calculation is auditable.
produce standards-compliant sustainability reports for small vessels	platform generates sustainability reports (fuel consumption, CO_2 emissions, water usage) that can be exported for regulatory filing	reports are produced within 5 s, contain all required fields, and correctly flag non-compliant periods	integration test that triggers report generation and then asserts three things: the file is produced within 5 seconds (measured wall-clock), a schema validator confirms all required fields are present (fuel

			consumption, CO_2, water usage, vessel ID, reporting period, etc.), and a second assertion flags any time windows where values exceeded the compliance threshold — confirming the non-compliant period detection logic fired correctly.

Methodology & Design

Suvion Systems consists of the following modules:

- Single-page Next.js application (dashboard) for the front-end portion of the app with unit tests for dashboard widgets and inputs
- Data Ingestion - Signal K receives NMEA0183/2000 packets over UDP/TCP. A Node.js adapter forwards parsed data to the dashboard via WebSocket and stores it in MongoDB. The service produces log files (e.g. from Socket.IO or a raw TCP listener) that captures socket and network connection
- Simulation - A mock NMEA generator (Python) streams synthetic data through the same Signal K pipeline, enabling demo and testing without hardware.
- Emissions Engine - A small TypeScript module applies IMO emission factors (fuel type × consumption) to compute CO_2. Results are cached and refreshed every minute. Known reference values are fed into the calculation module to validate the correctness of computations.
- Reporting - A REST API aggregates data over user-selected ranges and returns JSON formatted for export to CSV or PDF. The service includes integration tests to ensure correct report generation.

Potential future modules:

- Anomaly Detector - An isolation-forest model (Python) trained on a 24-hour baseline of simulated data.
- Optimization algorithms - seek patterns to recommend efficiency gains (i.e., "minimize overlap between generators when electrical load is low").
- Alerts service - alerts emitted when new readings exceed the model’s threshold, then sent to the front-end via WebSocket.
- Compliance automation

Scope

In Scope	Out of Scope	Future work (Out of Scope for MVP)
Simulated data input	Physical sensor installation on vessels	Anomaly detection (generator, water)
Real-time ingestion via Signal K	Integration with proprietary vessel management systems	Alerts and notifications
CO_2 emissions calculation using IMO formulas	User authentication/authorization	Standards-compliant report generation
Full system test on local MacOS and Linux machines	Cloud deployment (beyond a single local instance)	Optimization algorithms
Report generation with no regulatory acceptance	Regulatory certification beyond IMO formulas	

The project deliberately excludes hardware deployment, authentication mechanisms, commercial licensing and regulatory certification. Alerts notification and machine learning based anomaly detection and optimization are part of potential future work, and are also excluded from the initial scope.

The focus is on simulated sensor ingestion, emissions calculation using IMO formulas, and a clear visualization pipeline with reproducible results.

System Architecture & Design

Suvion Systems is composed of five loosely coupled modules that share a common data bus (WebSocket / REST). Data flows in one direction — from sensor source to storage to display — with privacy and timing controls applied at defined integration points.

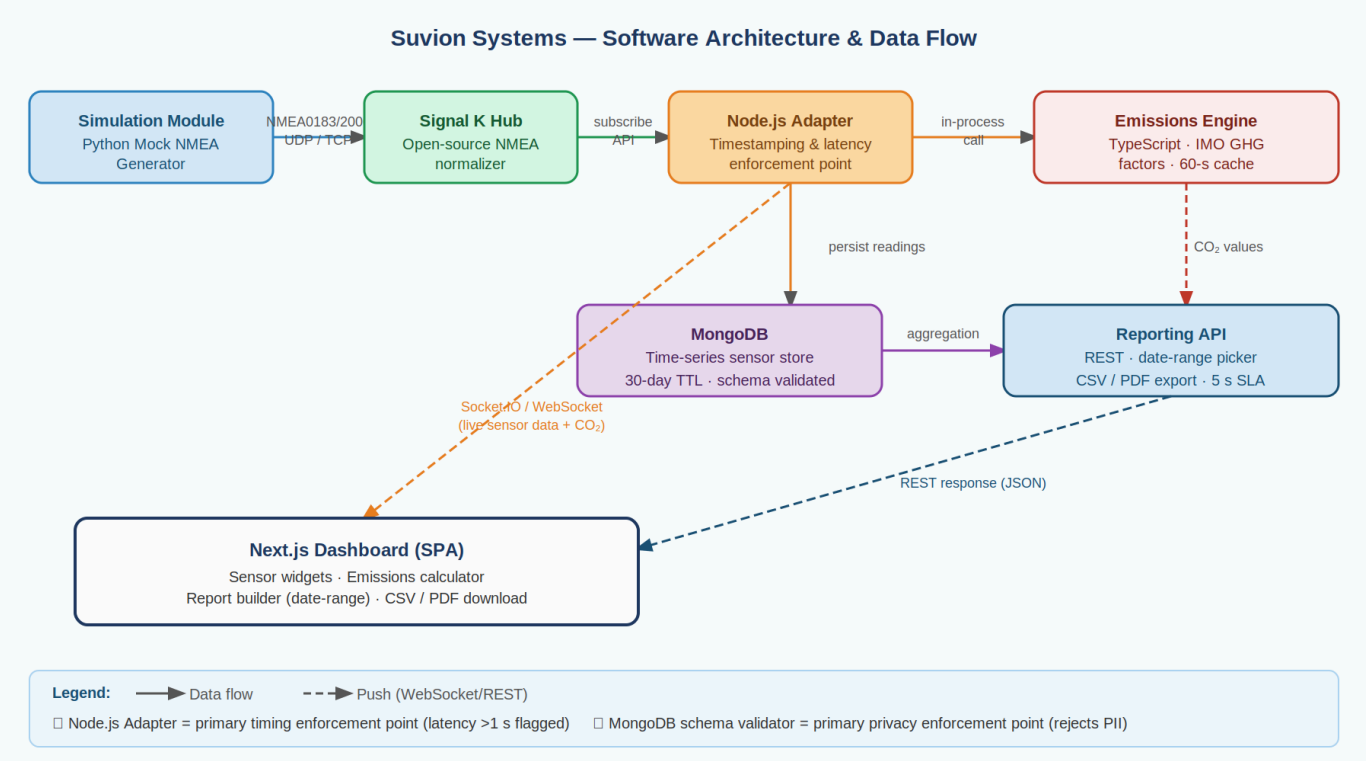


Figure 1 — Suvion Systems Software Architecture & Data Flow

Components

- **Dashboard (Next.js SPA).** Single-page application that renders sensor widgets, an emissions calculator, and a report builder with a date-range picker. Communicates with the back-end exclusively over WebSocket (live data) and REST (reports).
- **Signal K Hub.** Open-source server that receives NMEA0183 / NMEA2000 packets over UDP and TCP, normalises them into a standard key–value model, and exposes a subscribe API consumed by the Node.js adapter.
- **Node.js Adapter.** Bridges Signal K and the rest of the system. It forwards parsed readings to the dashboard via Socket.IO (WebSocket) and persists them to MongoDB. This is the primary timing enforcement point: messages are timestamped on receipt and any message that exceeds the 1-second latency budget is flagged in the connection log.
- **Emissions Engine (TypeScript).** Stateless calculation module that applies IMO GHG emission factors (fuel type × consumption rate) to produce CO_2 values. Results are cached in memory and refreshed every 60 seconds to avoid redundant recomputation while keeping displayed values current.
- **Reporting API (REST).** Aggregates stored sensor data over a user-selected date range and returns a JSON payload that the front-end renders and exports as CSV or PDF. Report generation must complete within 5 seconds (wall-clock); the API logs request and response timestamps for validation.

- **Simulation Module (Python).** Mock NMEA generator that streams synthetic packets through the same Signal K pipeline used by live hardware, enabling demo and test without physical sensors.

Data Flow

Simulation path: Python mock generator → Signal K (UDP/TCP) → Node.js adapter → Socket.IO → Dashboard widgets. In parallel, the adapter writes each reading to MongoDB with a UTC timestamp.
Reporting path: Dashboard date-range picker → HTTP GET /report → Reporting API → MongoDB aggregation → JSON payload → CSV / PDF download.
Emissions path: Node.js adapter → Emissions Engine (in-process call) → cached CO_2 value → Socket.IO → dashboard emissions widget. Cache is invalidated every 60 seconds or on fuel-type change.

State Transitions

The system operates in two top-level modes that share the same ingestion and storage pipeline:

- **Simulation mode (default).** The Python mock generator is the data source. The user can start or stop simulation from the dashboard; transition must complete in <2 seconds. Simulated scenarios include cruising, anchoring, and generator cycling.
- **Live mode (future).** Physical NMEA hardware replaces the mock generator; no other component changes. Out of scope for the MVP.

Within each mode the adapter transitions through: Idle → Connected (Signal K handshake) → Streaming (messages received) → Error (dropped connection, auto-reconnect attempted) → Idle.

Storage

All sensor readings are persisted in MongoDB as time-series documents (one document per reading, indexed on timestamp and stream type). The collection is capped at 30 days of data in accordance with the data-minimisation policy described in Ethics & Data Governance. No document may contain a crew name, vessel registration number, or any other PII field; this is enforced by a pre-insert schema validator and verified by the nightly privacy-check.py scan.

Privacy and Timing Enforcement Points

- **Node.js adapter (timing).** Timestamps every inbound message; latency exceeding 1 second triggers a log warning. This is the only place where end-to-end delivery time is measured.
- **MongoDB schema validator (privacy).** Rejects any insert that includes disallowed fields. Nightly privacy-check.py scans the collection and logs any anomaly to data-lifecycle.log.

- **Reporting API (timing).** Records wall-clock duration from request receipt to JSON response. Responses that exceed 5 seconds are flagged in reporting-api.log and counted as test failures.
- **30-day TTL job (privacy).** Automated script runs daily, permanently deletes documents older than 30 days, and appends a signed deletion record to data-lifecycle.log.

Success Metrics

Sensor data

Real-time sensor data (NMEA0183/2000) appears in the dashboard within <1 second of emission, with 100% accuracy (no loss/corruption/unit mismatches).

Test Procedure

- Use the mock NMEA generator (Python) to stream 500+ synthetic packets across 4 data streams (fuel flow, generator current, water usage, temperature) over a 24-hour replay period (simulating cruising, idling, generator surges).
- For each packet, record:
    - Timestamp of emission from the generator.
    - Timestamp of reception by Signal K (via UDP/TCP logs).
    - Timestamp of display in the dashboard (via WebSocket message logs).
- Validate that:
    - Dashboard update latency ≤ 1 second (95th percentile).
    - 100% match between simulated values and dashboard readings (e.g., fuel flow = 12.5 L/h → dashboard shows 12.5 L/h, no unit conversions like "L/h" vs "gal/min").

Sample Size

500+ synthetic NMEA packets per data stream (2,000 total).

Baseline

IMO NMEA data standard (valid packet formats, transmission rates).

Log Types

- Signal K connection logs (e.g., signal-k-server.log): Records UDP/TCP packet reception timestamps and payloads.
- WebSocket message logs (e.g., dashboard-client.log): Captures front-end data update events with timestamps.
- Mock generator logs: Tracks emission timestamps for baseline comparison.

Emissions Calculations

CO_2 emissions calculations differ from IMO reference values by <0.5% for all fuel types (diesel, marine gasoil, LNG) and consumption rates (0.1–500 L/h).

Test Procedure

- Create a test matrix of 100+ scenarios: 5 fuel types × 20 consumption rates (spanning idle to full load).
- For each scenario, compute expected CO_2 using IMO GHG Calculation Guidelines
- Feed scenario inputs into the TypeScript emissions engine and compare outputs to expected values.
- Replay scenarios over a weekly simulated operation period (7 days × 24 hours) to test stability.

Sample Size

100+ fuel-consumption scenarios.

Baseline

IMO emission factors (e.g., diesel = 2.68 kg CO_2/L, marine gasoil = 2.72 kg CO_2/L).

Log Types

- Emissions engine audit logs (e.g., emissions.log): Records input fuel type/consumption, output CO_2, and timestamp.
- Test result logs: Stores pass/fail for each scenario (error % vs. IMO reference).

Sustainability Reports

Reports are generated in <5 seconds, include all 12 required fields (vessel ID, reporting period, fuel consumed, CO_2 emitted, water usage, etc.), and correctly flag non-compliant periods (exceeding IMO carbon intensity thresholds).

Test Procedure

- Trigger report generation for 50+ date ranges (e.g., 1 day, 1 week, 1 month) using pre-simulated data.
- Measure:
    - Time to generate report (wall-clock time from API request to file download).
    - Completeness: Validate all 12 required fields via JSON schema (e.g., report-schema.json).
    - Compliance detection: For scenarios with fuel consumption > 150 L/h (arbitrary threshold for testing), confirm the report flags "non-compliant" with a timestamp.

- Replay tests over monthly report cycles (4 weeks) to ensure consistency.

Sample Size

50+ report requests across varying date ranges.

Baseline

IMO Environmental Contribution Report Template (MEPC.359(77))

Log Types

- API response logs (e.g., reporting-api.log): Records request/response timestamps and status codes.
- Schema validation logs: Confirms presence of required fields (e.g., "fuel_consumption_L" not missing).
- Compliance flag logs: Tracks when non-compliant periods are detected and included in reports.

Innovation & Differentiation

Suvion Systems is novel relative to the existing options described in Related Work in three concrete, measurable ways, each of which is within the approved project scope.

1. Simulation-First Accessibility [Experimental]

Every alternative in the market — commercial monitors (US$20–50 k), proprietary management suites, and spreadsheets — requires either expensive hardware, certified installation, or both before a user can interact with the system. Suvion Systems removes this barrier entirely: the mock NMEA generator starts streaming within <2 seconds, giving any operator a fully functional demo on a laptop with no hardware purchase. This is a qualitative change, not merely a cost reduction — the system can be evaluated, tested, and validated without a vessel.

- **Measurable comparison.** Simulation can be started and stopped in <2 seconds (target). Commercial monitors require physical installation measured in days.
- **Scope alignment.** Simulation mode is explicitly in scope and verified by the Simulation Mode success metric (<2 s start/stop, correct dashboard updates across cruising / anchoring / generator cycling scenarios).

2. Open-Source Stack for Zero Licensing Cost [Novel Application]

Commercial marine monitors carry a US$20–50 k capital cost plus proprietary licensing. Proprietary yacht management software adds recurring subscription fees and vendor lock-in. Suvion Systems is built entirely on open-source components (Signal K, Next.js, Node.js,

MongoDB Community, Python) with all emission factors sourced from publicly available IMO publications. The total software cost is $0.

- **Measurable comparison.** $0 software licensing cost vs. US$20–50 k for commercial alternatives. All five core components are verifiably open-source (OSI-approved licences).
- **Scope alignment.** Use of open-source software is explicitly stated as a goal in Goals & Proposed System and confirmed in the Feasibility & Build Plan.

3. Automated IMO Emissions Calculation with Auditable Accuracy [Exploratory]

Spreadsheets and manual logs require a captain to spend hours per season applying IMO GHG formulas by hand — a process that is error-prone and unauditable. Proprietary yacht management software offers limited or undisclosed environmental modules with no published accuracy targets. Suvion Systems automates the full calculation pipeline using openly sourced IMO GHG Guidelines (MEPC.361(78)) and targets a <0.5% error margin validated against 100+ reference scenarios. Every calculation is written to an emissions engine audit log (emissions.log) that cites the formula source, making each result independently reproducible.

- **Measurable comparison.** <0.5% emissions calculation error (target) vs. manual spreadsheet calculations with no documented error bound. Calculation results are produced in real time vs. hours of end-of-season manual effort.
- **Scope alignment.** The Emissions Calculations success metric (<0.5% error, 100+ scenarios across 5 fuel types and 20 consumption rates) directly verifies this claim within the approved project scope.

Risks & Mitigations

Risk	Impact	Likelihood	Mitigation
Data Quality – Simulated NMEA streams may not capture all real-world edge cases.	Medium	High	Use a diverse set of synthetic scenarios (cruising, idling, generator surge) and validate against known IMO emission factors.
Emission calculation accuracy	High	Low	Apply the equations of the IMO, and check comparison case studies with the reports from the research paper of the maritime branch. Document assumptions clearly.
Lack of domain expertise in maritime operations	Low	Low	Conduct a comprehensive study of public documents, including IMO guidelines, EU regulations, maritime

			engineering textbooks; verify assumptions with published research.

Testing & Validation Plan

The test suite covers four tiers: unit tests for isolated modules, integration tests for component interactions, end-to-end system tests for full pipeline validation, and a performance tier. All tests are automated and included in the submitted codebase (npm test / pytest). Results are logged to /test-results/.

Unit Tests — Emissions Engine

ID	Category	Test Description	Expected Result	Pass Criterion
UT-E01	Unit	Diesel 100 L/h input	CO_2 = 268.0 kg	|output − reference| / reference < 0.005
UT-E02	Unit	Marine gasoil 50 L/h input	CO_2 = 136.0 kg	Same error bound
UT-E03	Unit	LNG 200 L/h input	CO_2 = 449.6 kg	Same error bound
UT-E04	Unit	Unknown fuel type passed to engine	Exception thrown	Exception type and message match spec
UT-E05	Unit	Full matrix: 5 fuel types × 20 rates (100 scenarios)	All outputs within 0.5 % of IMO reference	All 100 assertions pass; MEPC.361(78) cited in test file
UT-E06	Unit	Cache hit after 59 s (same inputs)	Returns cached value	cache.hit counter increments; output identical
UT-E07	Unit	Cache miss after 61 s (TTL expired)	Recomputes and updates cache	cache.miss counter increments; fresh value stored

Unit Tests — Dashboard Widgets

ID	Category	Test Description	Expected Result	Pass Criterion
UT-W01	Unit	Fuel-flow widget receives {value:12.5, unit:"L/h"}	Widget renders "12.5 L/h"	Rendered text exact match; no unit conversion
UT-W02	Unit	WebSocket disconnects mid-stream	Widget shows "Reconnecting …" state	State label and CSS class match spec
UT-W03	Unit	Date-range picker: start > end	Report button disabled; error message shown	Button aria-disabled=true; error text visible
UT-W04	Unit	Simulation toggle clicked OFF → ON	UI updates to “Simulation active” within 500 ms	State change < 500 ms; label correct

Integration Tests — Data Ingestion

ID	Category	Test Description	Expected Result	Pass Criterion
IT-D01	Integration	Generator emits 500 packets across 4 streams	All 500 appear in MongoDB	MongoDB count = 500; 0 missing
IT-D02	Integration	500 packets; compare emission timestamp vs dashboard receipt	p95 latency ≤ 1 s	95th percentile of (receipt − emission) ≤ 1000 ms
IT-D03	Integration	Fuel-flow 12.5 L/h emitted; check	Dashboard receives exactly 12.5 L/h	JSON payload field match; no rounding or unit conversion

		dashboard payload		
IT-D04	Integration	Signal K connection dropped	Adapter reconnects within 5 s	connection.log shows reconnect event; subsequent packets received
IT-D05	Integration	PII-containing document sent to MongoDB (schema validator active)	Insert rejected	MongoDB WriteError; entry in data-lifecycle.log
IT-D06	Integration	2,000-packet load test (4 streams × 500)	p95 latency ≤ 1 s under load	No degradation vs IT-D02 baseline

Integration Tests — Reporting API

ID	Category	Test Description	Expected Result	Pass Criterion
IT-R01	Integration	1-day date range report request	Report JSON produced	Wall-clock < 5 s; all required fields present
IT-R02	Integration	1-month date range report request (worst-case aggregation)	Report JSON produced	Wall-clock < 5 s
IT-R03	Integration	Pre-seeded data with fuel > 150 L/h period	Report flags non-compliant period	non_compliant_windows array present with correct timestamps
IT-R04	Integration	50 report requests across varying date ranges	All 50 produced within SLA	All wall-clock times < 5 s; 100 % field completeness
IT-R05	Integration	Report export as CSV and as PDF	Files downloadable in both formats	Content-Type headers correct; files non-empty

End-to-End / System Tests

ID	Category	Test Description	Expected Result	Pass Criterion
E2E-01	System	Start simulation from dashboard UI	Simulation active state in < 2 s	All 4 sensor widgets show live values
E2E-02	System	Cruising scenario: diesel 45 L/h for 1 h	CO_2 widget ~120.6 kg	Widget value within 0.5 % of expected; continuously updating
E2E-03	System	Stop simulation from dashboard UI	Dashboard enters Idle state in < 2 s	Widgets stop updating
E2E-04	System	Generate 1-month report after 24 h simulated run	Report with all required fields; non-compliant periods flagged	Schema validator passes; download < 5 s
E2E-05	System	24-hour continuous simulated run (stability)	No memory leak, no crash, no data loss	Process memory stable (±10 %); 100 % packet retention
E2E-06	System	Run privacy-check.py on live MongoDB	No PII fields found	Script exits 0; clean scan logged

Deliverables

The following artefacts will be produced and submitted at project completion. All source deliverables will be hosted in a public GitHub repository.

#	Deliverable	Format / Location	Description & Acceptance Criteria
D-01	Source Code Repository	GitHub (public) /src	All six modules (Dashboard, Signal K config, Node.js Adapter, Emissions Engine, Reporting API, Simulation

			Module). README with setup, run, and test instructions. All dependencies pinned in package.json and requirements.txt.
D-02	Automated Test Suite	GitHub /tests — npm test + pytest	Unit tests (UT-E01–E07, UT-W01–W04), integration tests (IT-D01–D06, IT-R01–R05), E2E tests (E2E-01–E2E-06). All tests pass on a clean checkout.
D-03	Test Results Report	PDF — /test-results/report.pdf	Pass/fail summary for all test cases. For each failing test: root cause, fix applied, re-test result. Includes p95 latency measurements and emissions accuracy percentages.
D-04	Emissions Audit Logs	Text — /logs/emissions.log	Sample output from 100+ scenario run. Each entry: timestamp, fuel type, consumption rate, IMO emission factor, computed CO_2, MEPC.361(78) clause reference.
D-05	User Manual	PDF — /docs/user-manual.pdf	Installation guide (macOS & Linux); step-by-step startup; dashboard tour; how to start/stop simulation; how to generate and download a report; troubleshooting.
D-06	Architecture Documentation	PDF — /docs/architecture.pdf	System architecture diagram (Figure 1), data flow description, component responsibilities, and enforcement point definitions.
D-07	Privacy & Data Lifecycle Log	Text — /logs/data-lifecycle.log	Sample output from nightly privacy-check.py and 30-day TTL job. Each entry: timestamp, action, records affected.

Schedule & Milestones

The schedule below expands each phase into deliverables, risk triggers, and a buffer allocation.

Phase	Details
Weeks 1–2 | Foundation	Signal K running; mock generator emitting; dashboard skeleton deployed.
Weeks 3–5 | Core pipeline	Emissions engine.
Weeks 6–7 | Reporting	Reporting API / PDF export functional
Weeks 8–9 | Buffer	All unit & integration tests pass; fixing all bugs and issues
Weeks 10–11 | System test	performance tuning; documentation complete.
Week 12 | Delivery	Demo ready; final risk review/

Weeks 1-2: Setup Signal K, mock NMEA generator, basic dashboard skeleton.

Deliverables: working Signal K instance receiving NMEA0183 packets over UDP; mock NMEA generator producing at least 4 data streams (fuel flow, generator current, water usage, temperature); basic Next.js dashboard rendering one widget per stream with live WebSocket data. Risk trigger: if Signal K fails to parse a custom NMEA sentence by end of Week 1, switch to raw TCP listener and file a scope note.

Weeks 3-5: Emissions engine & real-time data pipeline.

Deliverables: TypeScript emissions engine; Node.js adapter. Risk trigger: if ±0.5% accuracy is not achieved by end of Week 4, extend engine work into the buffer in weeks 8–9 and defer Reporting API start by one week.

Weeks 6-7: Reporting API, export formats

Deliverables: REST Reporting AP; CSV and PDF export functional from the dashboard date-range picker. Risk trigger: if MongoDB aggregation for 1-month ranges exceeds 5 s, add a pre-aggregation index and extend optimization into weeks 8–9 buffer.

Weeks 8-9: Bug fixes, unit testing. Potentially: anomaly detection model and alert UI

Deliverables: all unit and integration tests passing. Weeks 8–9 serve as the primary schedule buffer: if the emissions engine or Signal K integration required remediation in weeks 3–7, this period absorbs it.

Weeks 10-11: Integration testing, performance tuning, documentation.

Performance tuning complete (p95 latency confirmed ≤1 s under 2 000-packet load; report generation confirmed ≤5 s for 1-month range).

Week 12: Demo prep, risk review, final polish.

Deliverables: live demo walkthrough

Assumptions & Constraints

The following assumptions have been made in scoping and designing this project. Each assumption is paired with the constraint it implies and the action taken if the assumption proves false.

Assumptions

- The Signal K open-source server (v2.x) correctly parses NMEA0183 and NMEA2000 packets over UDP/TCP without requiring custom protocol modifications. If false: fall back to a raw TCP listener and implement a custom parser; schedule impact absorbed in the weeks 8–9 buffer.
- IMO GHG Calculation Guidelines (MEPC.361(78)) are publicly available and provide unambiguous emission factors for diesel, marine gasoil, LNG, HFO, and MDO. If false: use MEPC.245(66) as a fallback; document the substitution.
- Simulated NMEA data (synthetic packets) is sufficient to validate all functional requirements. No physical vessel hardware is available or required. If false: the scope note in Section 4 (Scope) applies; live-hardware testing is out of scope for this project.
- The local development machines (macOS and Linux) provide adequate performance to run Signal K, MongoDB, the Next.js dashboard, and the Python generator concurrently without hardware-level bottlenecks. If false: reduce mock-generator packet rate from 1 Hz to 0.2 Hz and note the change in the test report.
- MongoDB Community Edition is sufficient for time-series storage at the volumes generated in this project (≤2 000 packets/test run, ≤30 days of data). No managed cloud database is required.
- The project's test environment has stable internet access for installing npm and pip packages during setup. All production dependencies will be pinned in package.json and requirements.txt before the integration test phase begins.

Constraints

- No physical hardware. The system is tested exclusively with simulated NMEA data. Hardware installation on a vessel is out of scope.

- No user authentication. The dashboard assumes all users are authorised crew members on a private local network. Authentication is out of scope for the MVP.
- Single local deployment. The system is deployed on one machine (MacOS or Linux). Multi-instance, clustered, or cloud deployments are out of scope.
- No regulatory certification. Reports are generated using IMO formulas but are not submitted to any regulatory body. The system does not guarantee acceptance of its outputs as standards-compliant documents.
- No PII. The system must not store or process any personally identifiable information. All sensor readings are numerical values with no operator, crew, or vessel identification beyond a configurable vessel_id string.
- Open-source dependencies only. All software components must carry an OSI-approved open-source licence. No proprietary SDKs, paid APIs, or commercial data feeds may be used.
- Time constraint: 12 weeks, single developer. Feature additions that cannot be completed and tested within this window are classified as future work (see Scope section).

Ethics & Data Governance

Given Suvion's focus on simulated data and prototype scope, ethics and data governance are centered on minimizing risk (no real user/PII data) and transparency (documented data lifecycle). Below is how retention, deletion, and privacy are verified:

Data Retention

- Policy: Simulated sensor data is retained for 30 days maximum (aligned with The General Data Protection Regulation (GDPR's) "data minimization" principle). No real vessel or crew data is stored.
- Verification: Automated scripts run daily to delete data older than 30 days. Logs of deletion events are stored in data-lifecycle.log for audit.

Data Deletion

- Policy: All test data is purged permanently upon project completion (or earlier at user request).
- Verification: A final "data wipe" script is documented and executed post-demo, with a cryptographically signed log confirming deletion.

Privacy Constraints

- Policy: No personal identifiable information (PII) or vessel operator data is collected. All "user" interactions are with simulated data (e.g., no login credentials, no crew names).
- Verification: Regular scans using privacy-check.py confirm no PII fields (e.g., email, vessel registration numbers) exist in the database (MongoDB).

Compliance

Documentation file is maintained, detailing:

- Data types stored (only simulated sensor readings: fuel flow, temperature—no PII).
- Retention/deletion workflows.
- Privacy impact assessment (PIA) confirming no real-user risk.

Feasibility & Build Plan

Why This Is Achievable in One Term

Feasibility	Details
Prior Experience	I have built small, fast web tools that ingest real-time data and expose dashboards.
Tech Stack	Next.js, Node.js, TypeScript, MongoDB, and Python are my core languages
Resources	All dependencies are open-source; IMO and EPA publish emission factors to the public. No external hardware or paid services are required.
Validation	Success metrics don't require vessel access. Calculations validate against published papers.

Canonical Terms

Term	Definition	Unit	Verification Method
Sensor Data Ingestion Latency & Accuracy	Time from sensor data emission (mock NMEA0183/2000 packet) to dashboard display, plus percentage of values matching simulated inputs (no loss/corruption/unit mismatches).	Seconds (latency: ≤1s target) / % (accuracy: 100% target)	Stream 500+ synthetic packets via mock NMEA generator; compare emission/dashboard timestamps (Signal K/WebSocket logs) and validate 100% value match (e.g., fuel flow = 12.5 L/h

			→ dashboard shows 12.5 L/h).
Emissions Calculation Error	Relative difference between Suvion's CO_2 emission output and IMO GHG Guidelines (MEPC.361(78)) reference values for fuel type/consumption scenarios.	% (error: ≤0.5% target)	Test 100+ scenarios (5 fuel types × 20 consumption rates: 0.1–500 L/h); compare Suvion outputs to IMO-calculated references using emissions engine audit logs (e.g., diesel emission factor = 2.68 kg CO_2/L).
Report Generation Time	Wall-clock time from API request (user selects date range) to report download (CSV/PDF).	Seconds (≤5s target)	Trigger 50+ report requests across date ranges (1 day/1 week/1 month); measure time via API response logs (reporting-api.log).
Report Field Completeness	Percentage of required fields (vessel ID, reporting period, fuel consumed (L), CO_2 emitted (kg), water usage (m^3)) present in generated reports.	% (100% target)	Validate reports against IMO-compliant JSON schema (report-schema.json); track missing fields via schema validation logs.
Compliance Detection Accuracy	Percentage of simulated non-compliant periods (exceeding IMO carbon intensity threshold: e.g., >150 L/h fuel consumption) correctly flagged in reports.	% (100% target)	Simulate 20+ threshold breaches; confirm reports flag "non-compliant" with timestamps via compliance flag logs.

Related Work

A search on GitHub and google reveals no repositories offering integrated yacht environmental monitoring combining fuel, emissions, water, and energy with optimization algorithms. This gap forces operators to choose between expensive proprietary solutions or manual tracking.

Existing solution	Limitation
Spreadsheets / Manual Logs	Time-consuming, error-prone, no real-time insight
Commercial Marine Monitors	High upfront cost (20–50 k), require certified installation, target large fleets
Proprietary Yacht Management Software	Limited environmental modules, no automated emissions calculations, no anomaly detection

None of the above options provide a single, web-based dashboard that ingests standard NMEA0183/ NMEA2000 data, automatically computes CO_2 emissions using IMO formulas and exports reports.

Suvion Systems differentiates itself from existing solutions through three measurable, scope-aligned design decisions that address the unmet needs of small vessel operators:

- Simulation-First Accessibility (Eliminates Hardware Barriers)
- Open-Source Stack for Affordability
- Automated IMO Compliance Automation (Eliminates Manual Error)